

Python Thread

Задание 1 (Формирование управляющего сигнала)

Выполнение текущего задания производить на собственных машинах. USB-камеру взять у одноклассников, в крайнем случае можно прийти и проверить работу алгоритма на Лаврентьева 6, камеру выдам.

При работе с ресурсами, которые должны явно освобождаться используйте идиому RALL. Т.е. напишите класс в конструкторе которого происходит инициализация, а в деструкторе освобождение ресурса.

Цель: чтения данных с датчиков и формирование картинки. Задержки должны быть минимальны.

Для всех датчиков:

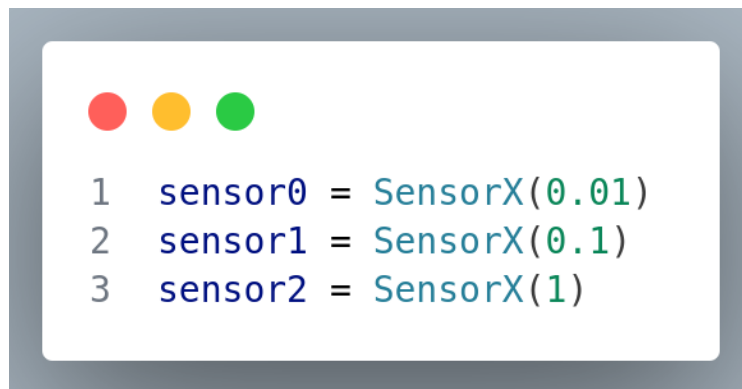
- 1) данные с датчиков нужно считывать постоянно, для получения самых последних значений;
- 2) каждый датчик обрабатывать в отдельном потоке;
- 3) обмен данными между потоками организовать через очереди (модуль queue).

Датчики состоят из usb-камеры (SensorCam) и класса SensorX. Для датчика usb-камеры написать класс SensorCam, для остальных датчиков — SensorX. Они должны наследоваться от класса Sensor:

```
1 class Sensor:
2     def get(self):
3         raise NotImplementedError("Subclasses must implement method get()")
```

```
1 class SensorX(Sensor):
2     '''Sensor X'''
3     def __init__(self, delay: float):
4         self._delay = delay
5         self._data = 0
6
7
8     def get(self) -> int:
9         time.sleep(self._delay)
10        self._data += 1
11        return self._data
```

Использовать три датчика SensorX:



Т.е. датчики работают с частотой 100 Hz, 10 Hz и 1Hz.

Для взаимодействия с камерой использовать OpenCV.

В основном потоке получать данные со всех датчиков (SensorCam и SensorX), с частотой отображения картинке в окне. Выводить изображение с камеры и датчиков в окно, на экране монитора (релизовать класс WindowImage, он отвечает за отображения картинки). Пример вывода:



При запуске программы производить парсинг командной строки, используя модуль `argparse`. Опции:

- 1) имя камеры в системе
- 2) желаемое разрешение камеры
- 3) частота отображения картинки.

При нажатии на клавишу «q» программа должна безопасно завершать работу, освобождая все ресурсы. Формировать картинку можете отдельной функцией или наследоваться от класса `WindowImage`. Если новых данных нет для текущего датчика, то использовать его предыдущее значение.

Класс `SensorCam`

Класс `SensorCam` должен иметь следующие методы:

- 1) конструктор `__init__`
- 2) метод `get`, для получения кадра с камеры
- 3) деструктор `__del__`, для корректного завершения работы с камерой (например `cam.release()`)

Конструктор класса `Camera` принимает следующие параметры:

- 1) имя камеры в системе (например `/dev/video0`);
- 2) желаемое разрешение камеры (например `1280x720`).

При ошибках чтения или инициализации, выводить соответствующие сообщения в `log file` (лог файлы должны находиться в папке `log`, текущего проекта, для логирования использовать модуль `logging`), и корректно завершать программу если ошибку не удалось исправить. Примеры ошибок:

- 1) камеры с таким именем нет в системе;
- 2) камера внезапно вышла из строя (выдернули из `usb`).

Класс `WindowImage`

Класс `WindowImage` должен иметь следующие методы:

- 1) конструктор `__init__`
- 2) метод `show(img)`, чтобы отобразить картинку окне

3) деструктор `__del__`, для корректного завершения работы с окном (например `cv2.destroyAllWindows()`)

Конструктор класса принимает следующие параметры:

1) частота отображения картинки;

При ошибках вывода или инициализации, выводить соответствующие сообщения в log file (лог файлы должны находиться в папке log, текущего проекта, для логирования использовать модуль logging), и корректно завершать программу если ошибку не удалось исправить.